

R Programming

A Friendly Guide for Beginners and Beyond

Covers R 4.x, the Tidyverse, ggplot2, base R, and statistics. 2025 edition.

Prepared by

Lackson David

lacksinho@gmail.com

+255 746 910 200 / +255 686 609 294

ladam.dev

What's Inside

1. Getting to Know R
2. Setting Up Your Tools
3. The Basics of R Syntax
4. Data Types
5. Data Structures
6. Vectors, Up Close
7. Control Flow
8. Functions
9. Working with Data Frames
10. Reading and Writing Files
11. Cleaning and Shaping Data with dplyr and tidyr
12. Making Charts with ggplot2
13. Working with Text
14. Statistics in R
15. Good Habits and Handy Tips

A Note Before You Start

I put this guide together after spending a lot of time helping people get past the early confusion with R, the kind that comes from staring at an error message that makes no sense yet. Nothing here is theory for theory's sake. Every example is something you would actually type while working with real data.

Go through it in order if you are new to R, or jump straight to whatever section you need if you already know the basics. Either way works.

If something here helped you out, or if you spot a mistake, feel free to reach me at the contact details on the cover page.

Lackson David

1. Getting to Know R

R is a free programming language built for working with numbers, statistics, and charts. Two professors, Ross Ihaka and Robert Gentleman, created it at the University of Auckland back in 1993. Since then it has become one of the most popular tools in data science, research, and academia.

Why People Choose R

- It was built specifically for statistics and data analysis, so it does this job well.
- There are more than 20,000 add on packages available through CRAN, R's package library.
- Its charts, especially with the ggplot2 package, look great with very little effort.
- There is a large, friendly community and plenty of free tutorials online.
- It is used widely in healthcare, finance, academic research, and more.
- It plays nicely with Python, SQL, Spark, and other tools you may already use.

R Compared to Python

People often ask how R stacks up against Python. Here is a quick side by side look:

Feature	R	Python
Best for	Statistics and data analysis	General programming and machine learning
Charts	ggplot2 (very strong)	matplotlib or seaborn
Learning curve	Moderate	Gentle
Data tables	Built in	Needs the pandas package
Used mostly by	Academics and researchers	Software engineers and industry teams
Package source	CRAN	PyPI or conda

2. Setting Up Your Tools

Getting started with R takes just two installs: R itself, and RStudio, the program most people use to write R code.

Step 1: Install R

- Go to cran.r-project.org and download the version for your computer (Windows, macOS, or Linux).
- Run the installer and keep the default settings.
- To check it worked, open a terminal and type `r --version`.

Step 2: Install RStudio

RStudio is the recommended home base for writing R code. It is free and gives you a script editor, a console, a list of your variables, and a place to view plots, all in one window.

- Go to posit.co/downloads and download RStudio Desktop.

Step 3: Install a Few Key Packages

Packages add extra abilities to R. Here are the ones you will reach for the most:

```
# Run these once in the R console
install.packages("tidyverse") # data wrangling plus ggplot2
install.packages("ggplot2")   # charts
install.packages("dplyr")     # shaping data
install.packages("readr")     # fast CSV reading
install.packages("tidyr")     # tidying data

# Load a package (do this every session)
library(tidyverse)

# See what is already installed
installed.packages()[, "Package"]
```

Tip: You only need to install a package once. After that, just load it with `library()` each time you start a new R session.

Finding Your Way Around RStudio

Panel	Where it is	What it's for
Script editor	Top left	Write and save your R scripts (.R files)
Console	Bottom left	Run code right away and see the results
Environment	Top right	See your variables and data at a glance
Files and plots	Bottom right	Browse files, view charts, read help pages

3. The Basics of R Syntax

R's syntax is short and readable once you get used to a few habits. Here are the building blocks you will use every single day.

Assigning Values to Variables

In R, the arrow <- is the usual way to assign a value to a name (though = also works).

```
name <- "Alice"      # a piece of text
age <- 30             # a whole number
height <- 5.7         # a decimal number
is_student <- TRUE    # true or false

# Printing a variable
print(name)          # [1] "Alice"
name                 # does the same thing as print(name)
cat("Hello,", name, "\n") # Hello, Alice

# You can assign to several names at once
x <- y <- z <- 0     # all three are now 0
```

Doing Math

```
x <- 10; y <- 3
x + y      # 13
x - y      # 7
x * y      # 30
x / y      # 3.333...
x %% y      # 1, the remainder after dividing
x %/% y     # 3, division rounded down
x ^ y      # 1000, x to the power of y
```

Comparing and Combining Logic

```
5 > 3      # TRUE
5 == 5     # TRUE, this checks equality
5 != 4     # TRUE, means "not equal"
5 >= 10    # FALSE

TRUE & FALSE # FALSE, checks every value (AND)
TRUE | FALSE # TRUE, checks every value (OR)
!TRUE        # FALSE, flips the value
TRUE && FALSE # FALSE, stops at the first answer (AND)
TRUE || FALSE # TRUE, stops at the first answer (OR)

# Checking what type something is
is.numeric(3.14) # TRUE
is.character("hi") # TRUE
is.logical(FALSE) # TRUE
```

Writing Comments

A comment is a note in your code that R ignores when it runs. Start a line with # to write one. R does not have a way to write a comment across several lines, so just put # at the start of each line.

```
# This is a comment  
x <- 42  # you can also add a comment after code
```

4. Data Types

R has six basic data types. Knowing them well will save you a lot of head scratching, because most strange errors in R come from a value being a different type than you expected.

The Six Types

```
# 1. Numeric (a decimal number, the default for numbers)
x <- 3.14
class(x)      # "numeric"
typeof(x)     # "double"

# 2. Integer (a whole number, add an L to make one)
n <- 42L
class(n)      # "integer"

# 3. Character (text)
name <- "R Programming"
nchar(name)   # 13, the number of characters

# 4. Logical (true or false)
flag <- TRUE
flag <- FALSE
class(flag)   # "logical"

# 5. Complex (numbers with a real and imaginary part)
z <- 3 + 2i
Re(z); Im(z)  # 3, 2

# 6. Raw (raw bytes, you will rarely need this)
r <- charToRaw("A") # 41
```


Checking and Changing Types

```
# Checking a type
is.numeric("3")    # FALSE, it is text, not a number
is.character("3")  # TRUE

# Converting between types
as.numeric("3.14") # 3.14
as.integer(3.9)    # 3, it cuts off the decimal, it does not round
as.character(100)  # "100"
as.logical(0)      # FALSE, since 0 means false
as.logical(1)      # TRUE, anything else means true

# Special values you will run into
NA      # a missing value
NULL    # nothing at all, an empty object
NaN     # not a number, like the result of 0/0
Inf     # infinity, like the result of 1/0
-Inf    # negative infinity

is.na(NA)      # TRUE
is.null(NULL)  # TRUE
is.nan(NaN)    # TRUE
```

Tip: Use `class()` to see what type R thinks a value is. Use `typeof()` if you need to see how it is stored internally.

5. Data Structures

A data structure is just a container for your values. R gives you several to choose from, and picking the right one makes your code easier to read and faster to run.

Structure	Shape	Good for
Vector	One dimension, same type	The most common structure, the basis for everything else
Matrix	Two dimensions, same type	Tables of numbers, linear algebra
Array	Many dimensions, same type	Data with more than two dimensions
List	One dimension, mixed types	A flexible container, similar to JSON
Data frame	Two dimensions, mixed column types	Tables of data, like a spreadsheet
Factor	Categories	Labels or groups, like Male and Female

A Quick Look at Each One

```
# Vector
v <- c(10, 20, 30, 40)

# Matrix
m <- matrix(1:6, nrow = 2, ncol = 3)
#      [,1] [,2] [,3]
# [1,]    1    3    5
# [2,]    2    4    6

# List
person <- list(name = "Alice", age = 30, scores = c(95, 87, 92))
person$name      # "Alice"
person[["age"]]  # 30

# Data frame
df <- data.frame(
  name = c("Alice", "Bob", "Carol"),
  age = c(30, 25, 35),
  score = c(95.0, 87.5, 92.3)
)
df$name      # Alice Bob Carol
df[1, ]      # the first row
df[, "age"]  # the age column

# Factor (for categories)
gender <- factor(c("M", "F", "M", "F", "M"))
levels(gender) # "F" "M"
table(gender)  # F: 2, M: 3
```

6. Vectors, Up Close

Vectors are the heart of R. Even a single number like 42 is technically a vector that happens to hold one value.

Making Vectors

```
# c() combines values into a vector
nums <- c(1, 2, 3, 4, 5)
words <- c("apple", "banana", "cherry")

# Sequences
1:10          # 1 2 3 4 5 6 7 8 9 10
seq(0, 1, by = 0.25) # 0.00 0.25 0.50 0.75 1.00
seq(0, 1, length.out=5) # same result, five evenly spaced numbers

# Repeating values
rep(0, times = 5)      # 0 0 0 0 0
rep(c(1,2), times = 3) # 1 2 1 2 1 2
rep(c(1,2), each = 3)  # 1 1 1 2 2 2
```

Vector Math Without Writing a Loop

One of R's best features is that math on a vector applies to every value at once, no loop required.

```
v <- c(1, 2, 3, 4, 5)
v + 10 # 11 12 13 14 15, adds 10 to each value
v * 2  # 2 4 6 8 10
v ^ 2  # 1 4 9 16 25
sqrt(v) # 1.00 1.41 1.73 2.00 2.24

# Two vectors line up value by value
a <- c(1, 2, 3)
b <- c(10, 20, 30)
a + b # 11 22 33
a * b # 10 40 90
```

Picking Out Values

```
v <- c(10, 20, 30, 40, 50)
v[1]          # 10, R counts from 1, not 0
v[c(1,3)]     # 10 30
v[2:4]        # 20 30 40
v[-1]         # 20 30 40 50, drops the first value
v[v > 25]     # 30 40 50, only values over 25

# Naming values makes them easier to find later
scores <- c(Alice = 95, Bob = 87, Carol = 92)
scores["Alice"]      # 95
scores[c("Alice","Carol")] # 95 92
```

Handy Functions for Vectors

```
v <- c(4, 1, 7, 2, 9, 3)
length(v)    # 6, how many values
sum(v)       # 26
mean(v)      # 4.33
median(v)    # 3.5
max(v); min(v) # 9, 1
range(v)     # 1 9
var(v)       # variance
sd(v)        # standard deviation
sort(v)      # 1 2 3 4 7 9
rev(v)       # 3 9 2 7 1 4
which(v > 5)  # 3 5, the positions where this is true
any(v > 8)    # TRUE
all(v > 0)    # TRUE
```

7. Control Flow

Control flow is how you tell R to make a decision or repeat a task.

if, else if, and else

```
score <- 75
if (score >= 90) {
  grade <- "A"
} else if (score >= 80) {
  grade <- "B"
} else if (score >= 70) {
  grade <- "C"
} else {
  grade <- "F"
}
cat("Grade:", grade)    # Grade: C

# ifelse() does the same thing across a whole vector at once
x <- c(1, -2, 3, -4)
ifelse(x > 0, "positive", "negative")
# "positive" "negative" "positive" "negative"
```

The for Loop

```
# A basic loop
for (i in 1:5) {
  cat(i, " ")    # 1 2 3 4 5
}

# Looping over a vector of words
fruits <- c("apple", "banana", "cherry")
for (fruit in fruits) {
  cat("I like", fruit, "\n")
}

# Looping with a position number
for (i in seq_along(fruits)) {
  cat(i, ":", fruits[i], "\n")
}
```

while and repeat

```
# while keeps going as long as the condition is true
count <- 1
while (count <= 5) {
  cat(count, "")
  count <- count + 1
} # 1 2 3 4 5

# repeat runs forever until you tell it to stop with break
n <- 1
repeat {
  cat(n, "")
  n <- n + 1
  if (n > 5) break
}

# next skips ahead to the following loop step
for (i in 1:10) {
  if (i %% 2 == 0) next # skip even numbers
  cat(i, "") # 1 3 5 7 9
}
```

Tip: Loops work fine, but when you are working with a vector, a vectorized approach like `ifelse()` or `apply()` is usually faster and shorter to write.

8. Functions

A function is a reusable chunk of code. In R, functions are treated like any other value, so you can pass them around, store them in a variable, or return one from another function.

Writing and Calling a Function

```
# A basic function
greet <- function(name) {
  message <- paste("Hello,", name, "!")
  return(message)
}
greet("Alice")    # "Hello, Alice !"

# Giving an argument a default value
power <- function(base, exponent = 2) {
  return(base ^ exponent)
}
power(3)           # 9, uses the default exponent of 2
power(3, 3)        # 27, overrides the default
power(base = 2, exponent = 10) # 1024, using named arguments
```

Returning More Than One Value

A function can only return one object, but that object can be a list holding several results.

```
stats_summary <- function(x) {
  list(
    mean = mean(x),
    median = median(x),
    sd = sd(x),
    n = length(x)
  )
}
result <- stats_summary(c(4, 8, 15, 16, 23, 42))
result$mean    # 18
result$sd      # 13.49...
```

The apply Family: Loops Without Writing Loops

```
# sapply applies a function and simplifies the result
nums <- list(a = 1:5, b = 6:10, c = 11:15)
sapply(nums, mean)    # a = 3, b = 8, c = 13

# lapply always gives back a list
lapply(1:3, function(x) x^2)    # a list holding 1, 4, 9

# vapply is a safer sapply, since it checks the result type
vapply(1:5, function(x) x*2, numeric(1))    # 2 4 6 8 10

# tapply applies a function within groups
scores <- c(90, 85, 78, 92, 88)
groups <- c("A", "B", "A", "B", "A")
tapply(scores, groups, mean)    # A: 85.33, B: 88.5

# Map applies a function across two lists, value by value
Map("+", 1:3, 4:6)    # 5, 7, 9
```

Tip: Learning `sapply()` and `lapply()` first will let you replace most of the loops you would otherwise write.

9. Working with Data Frames

A data frame is R's go to structure for analysis. Think of it as a table, similar to a spreadsheet, where each column can hold a different type of data.

Creating and Inspecting a Data Frame

```
df <- data.frame(
  name = c("Alice", "Bob", "Carol", "Dave"),
  age = c(30, 25, 35, 28),
  score = c(92.5, 87.0, 95.0, 78.5),
  passed = c(TRUE, TRUE, TRUE, FALSE)
)

nrow(df)      # 4 rows
ncol(df)      # 4 columns
dim(df)       # 4 4
str(df)       # a quick overview of the structure
summary(df)   # a statistical summary
head(df, 2)   # the first 2 rows
tail(df, 2)   # the last 2 rows
colnames(df)  # the column names
```

Reading Values Out

```
df$name      # the name column
df[["age"]]  # the age column
df[1, ]      # the first row
df[, "score"] # the score column
df[1, "name"] # "Alice", a single cell
df[df$passed, ] # only the rows where passed is TRUE
```

Changing a Data Frame

```
# Add a column
df$grade <- ifelse(df$score >= 90, "A",
                  ifelse(df$score >= 80, "B", "C"))

# Add a row
new_row <- data.frame(name = "Eve", age = 22,
                      score = 88.0, passed = TRUE, grade = "B")
df <- rbind(df, new_row)

# Remove a column
df$passed <- NULL

# Remove the first row
df <- df[-1, ]

# Rename a column (the second one, in this case)
colnames(df)[2] <- "years"

# Filter rows
young <- df[df$years < 30, ]

# Sort by score, highest first
df_sorted <- df[order(df$score, decreasing = TRUE), ]
```

10. Reading and Writing Files

Most real analysis starts by loading data from a file and ends by saving your results. Here is how to handle the most common formats.

CSV Files

```
# Base R
df <- read.csv("data.csv")
df <- read.csv("data.csv", stringsAsFactors = FALSE, na.strings =
c("", "NA", "N/A"))

# readr, from the tidyverse, is faster and smarter about types
library(readr)
df <- read_csv("data.csv")          # figures out column types on its own
df <- read_csv("data.csv", skip = 2) # skip the first 2 rows
df <- read_csv("data.csv", n_max = 1000) # only read the first 1000 rows

# Writing a CSV
write.csv(df, "output.csv", row.names = FALSE)
write_csv(df, "output.csv") # the readr version, no row names by default
```

Other File Types

```
# Excel files, using the readxl package
library(readxl)
df <- read_excel("data.xlsx")
df <- read_excel("data.xlsx", sheet = "Sheet2", range = "A1:D100")

# JSON files, using the jsonlite package
library(jsonlite)
data <- fromJSON("data.json")
toJSON(df, pretty = TRUE)

# Tab separated files
df <- read.delim("data.tsv")
df <- read_tsv("data.tsv") # readr version

# R's own binary format, the fastest way to save an object
saveRDS(df, "data.rds")
df <- readRDS("data.rds")

# Saving several objects at once
save(df, model, file = "workspace.RData")
load("workspace.RData")
```

Working with Folders and File Paths

```
getwd()           # see your current working directory
setwd("/path/to/project") # change it
file.exists("data.csv") # check whether a file exists
list.files(".")    # list everything in the current folder
list.files(".", pattern="*.csv") # list only the CSV files
```

Tip: Use an RStudio Project (.Rproj file) for each project. It sets your working directory for you, so you can use short, relative file paths instead of long, hardcoded ones.

11. Cleaning and Shaping Data with dplyr and tidyr

dplyr and tidyr are two tidyverse packages that make working with data feel natural. They use the pipe operator, written as %>% or |>, to chain steps together in the order you would describe them out loud.

```
library(dplyr)
library(tidyr)
```

The Five Core dplyr Verbs

```
# Sample data
df <- data.frame(
  name = c("Alice", "Bob", "Carol", "Dave", "Eve"),
  dept = c("HR", "IT", "IT", "HR", "Finance"),
  sal = c(55000, 72000, 68000, 60000, 80000),
  years = c(3, 5, 4, 2, 7)
)

# 1. filter() keeps rows that match a condition
df |> filter(dept == "IT")
df |> filter(sal > 65000, years >= 4)

# 2. select() keeps or drops columns
df |> select(name, sal)
df |> select(-years)           # drop years
df |> select(starts_with("s")) # columns starting with "s"

# 3. mutate() adds or changes columns
df |> mutate(
  sal_k = sal / 1000,
  seniority = ifelse(years >= 5, "Senior", "Junior")
)

# 4. arrange() sorts rows
df |> arrange(sal)           # smallest to largest
df |> arrange(desc(sal))     # largest to smallest
df |> arrange(dept, desc(sal)) # sort by department, then salary

# 5. summarise(), usually paired with group_by(), totals things up
df |>
  group_by(dept) |>
  summarise(
    avg_sal = mean(sal),
    count = n(),
    max_sal = max(sal)
  )
```

Chaining Several Steps Together

```
# Find the top earning IT employee with 4 or more years
df |>
  filter(dept == "IT", years >= 4) |>
  arrange(desc(sal)) |>
  slice(1) |>
  select(name, sal)
```

Reshaping Data with tidyr

```
# pivot_longer turns wide data into long data
wide <- data.frame(name = c("Alice","Bob"),
                   math = c(90,85), science = c(88,92))

long <- wide |>
  pivot_longer(cols = c(math, science),
               names_to = "subject",
               values_to = "score")

# name  subject  score
# Alice  math    90
# Alice  science 88
# Bob    math    85  ...

# pivot_wider does the opposite, long back to wide
long |>
  pivot_wider(names_from = subject, values_from = score)

# drop_na removes rows with missing values
df |> drop_na()
df |> drop_na(sal)  # only check the sal column

# fill copies a value down (or up) to replace NA
df |> fill(dept, .direction = "down")
```

12. Making Charts with ggplot2

ggplot2 builds charts in layers. You start with your data, add what should go on each axis, then add the type of chart, and finally add labels and styling on top.

```
library(ggplot2)
```

The Basic Pattern

```
# Template
ggplot(data = df, aes(x = col1, y = col2)) +
  geom_something() +      # the type of chart
  labs(...) +             # titles and axis labels
  theme_something()       # overall styling
```

Common Chart Types

```
df <- mtcars    # a dataset that comes built into R

# 1. Scatter plot
ggplot(df, aes(x = wt, y = mpg)) +
  geom_point(size = 3) +
  labs(title = "MPG vs Weight", x = "Weight", y = "Miles per Gallon") +
  theme_minimal()

# 2. Line chart
ggplot(economics, aes(x = date, y = unemploy)) +
  geom_line(linewidth = 1) +
  labs(title = "US Unemployment Over Time")

# 3. Bar chart
ggplot(df, aes(x = factor(cyl))) +
  geom_bar() +
  labs(title = "Car Count by Cylinders", x = "Cylinders")

# 4. Histogram
ggplot(df, aes(x = mpg)) +
  geom_histogram(bins = 10) +
  labs(title = "MPG Distribution")

# 5. Box plot
ggplot(df, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot() +
  labs(x = "Cylinders", y = "MPG")

# 6. Grouped bar chart
ggplot(df, aes(x = factor(cyl), fill = factor(am))) +
  geom_bar(position = "dodge") +
  labs(x = "Cylinders", fill = "Transmission")
```

Adding Polish

```
ggplot(df, aes(x = wt, y = mpg, color = factor(cyl))) +  
  geom_point(size = 3, alpha = 0.7) +      # alpha makes points see through  
  geom_smooth(method = "lm", se = FALSE) +  # adds a trend line  
  labs(  
    title = "MPG vs Weight by Cylinders",  
    subtitle = "mtcars dataset",  
    x = "Weight (1000 lbs)",  
    y = "Miles Per Gallon",  
    color = "Cylinders"  
  ) +  
  theme_minimal() +  
  theme(  
    plot.title = element_text(size = 14, face = "bold"),  
    legend.position = "bottom"  
  )  
  
# Saving your chart to a file  
ggsave("my_plot.png", width = 8, height = 5, dpi = 300)
```


13. Working with Text

R has built in tools for handling text, and the stringr package, part of the tidyverse, adds a more consistent set of functions on top.

```
library(stringr)
```

Length, Case, and Trimming

```
s <- " Hello, R World! "
```

nchar(s)	# 20, base R
str_length(s)	# 20, stringr
toupper(s)	# " HELLO, R WORLD! "
tolower(s)	# " hello, r world! "
str_to_title(s)	# " Hello, R World! "
trimws(s)	# "Hello, R World!" (base R)
str_trim(s)	# same result, stringr
str_squish(s)	# also removes extra spaces between words

Cutting, Splitting, and Joining Text

```
substr("Hello World", 1, 5) # "Hello"
str_sub("Hello World", 1, 5) # "Hello"
str_sub("Hello World", -5) # "World", counting from the end

strsplit("a,b,c", ",")[[1]] # "a" "b" "c"
str_split("a,b,c", ",")[[1]] # same result

paste("Hello", "World") # "Hello World"
paste0("file", 1:3, ".csv") # "file1.csv" "file2.csv" "file3.csv"
str_c("Hello", "World", sep = " ") # "Hello World"
str_c(c("a","b","c"), collapse = "-") # "a-b-c"
```

Finding, Matching, and Replacing

```
str_detect("banana", "ana") # TRUE, the pattern was found
str_starts("Hello", "He") # TRUE
str_ends("Hello", "lo") # TRUE
grep("^a", c("apple","Avocado","banana"), value = TRUE) # "apple"

str_replace("Hello World", "World", "R") # "Hello R"
str_replace_all("aababab", "ab", "X") # "aXXX"
gsub("\\s+", "_", "Hello World") # "Hello_World"

# Formatting numbers as text
sprintf("Pi = %.2f", pi) # "Pi = 3.14"
formatC(12345.6, format = "f", digits = 1) # "12345.6"
```

14. Statistics in R

Statistics is what R was built for, so the most common tests and summaries are already built in, with no extra packages needed.

Describing a Set of Numbers

```
x <- c(23,45,12,67,34,89,11,56,78,43)

mean(x)      # 45.8
median(x)    # 44.0
var(x)       # 601.96
sd(x)        # 24.53
range(x)     # 11 89
quantile(x)  # 0%: 11, 25%: 22.25, 50%: 44, 75%: 63.25, 100%: 89
IQR(x)       # 41, the spread of the middle half of the data
summary(x)   # a compact view of everything above
cor(x, y)    # Pearson correlation between two vectors
```

Testing a Hypothesis

```
# One sample t-test: is the mean really 50?
t.test(x, mu = 50)

# Two sample t-test: compare two groups
group_a <- c(82, 79, 88, 91, 75)
group_b <- c(70, 74, 68, 80, 72)
t.test(group_a, group_b)           # the default, a Welch t-test
t.test(group_a, group_b, paired = TRUE) # use this if the groups are paired

# Chi-square test, for checking independence
observed <- matrix(c(50,30,20,40), nrow = 2)
chisq.test(observed)

# Correlation test
cor.test(group_a, group_b, method = "pearson")

# Shapiro-Wilk test, for checking if data looks roughly normal
shapiro.test(x) # a p-value above 0.05 usually suggests it is normal
```

Linear Regression

```
# A simple model with one predictor
model <- lm(mpg ~ wt, data = mtcars)
summary(model)    # shows coefficients, R-squared, and p-values

# A model with several predictors
model2 <- lm(mpg ~ wt + hp + cyl, data = mtcars)
summary(model2)

# Making a prediction
new_data <- data.frame(wt = 3.0, hp = 110, cyl = 6)
predict(model2, new_data)    # the predicted mpg

# Checking the model
confint(model2)    # confidence intervals for each coefficient
AIC(model2)        # a score for comparing models
plot(model2)       # four diagnostic charts
```

Tip: Before trusting a regression model, check that the residuals look roughly normal (qqnorm), that they spread out evenly across predicted values, and that no single point is throwing things off.

15. Good Habits and Handy Tips

A few small habits go a long way toward making your R code easier to read, easier to share, and easier to trust later on.

Use RStudio Projects

Create one .Rproj file per project. It sets your working directory automatically, so your file paths stay short and work the same way on any computer.

Stick to a naming style

Use snake_case for variables and functions, such as my_data or calc_mean. Use UPPER_CASE for constants, such as MAX_SIZE.

Favor vectorized code over loops

An operation like `x + 10` runs across a whole vector at once and is usually much faster than writing a loop by hand.

Use R Markdown or Quarto

These let you combine code, results, and writing in one file, then export the whole thing as HTML, PDF, or Word.

Keyboard Shortcuts Worth Knowing (RStudio)

Action	Windows / Linux	macOS
Run current line	Ctrl+Enter	Cmd+Enter
Assignment arrow <-	Alt+-	Option+-
Pipe operator >	Ctrl+Shift+M	Cmd+Shift+M
Comment or uncomment	Ctrl+Shift+C	Cmd+Shift+C
Find and replace	Ctrl+H	Cmd+H
Format code	Ctrl+Shift+A	Cmd+Shift+A
New R script	Ctrl+Shift+N	Cmd+Shift+N

Keep learning at r-project.org, tidyverse.org, r4ds.hadley.nz, and ggplot2.tidyverse.org